

Algorithms

Complete MCQ-Oriented Notes

IBPS SO (IT Officer) Examination - Complete Preparation Series

100% Syllabus Coverage - MCQ-Oriented - Zero Filler

Compiled by Asabhya Maanav

Table of Contents

Part A1 - Growth of Functions & Asymptotic Analysis	6
Why Asymptotic Analysis	6
Big-O (Upper Bound)	6
Big-Omega (Lower Bound)	6
Big-Theta (Tight Bound)	6
Little-o and Little-omega	6
Ordering of Common Functions	6
Comparing Growth Rates	7
Properties of Asymptotic Notation	7
Part A2 - Types of Analysis	8
Best, Average, Worst Case	8
Time vs Space Complexity	8
Counting Basic Operations	8
Amortized Analysis (Awareness)	8
Part A3 - Recurrence Relations	9
Forming a Recurrence	9
Substitution Method	9
Recursion Tree Method	9
Master Theorem	9
Common Recurrences Cheat Sheet	9
Part B - Searching	11
Linear / Sequential Search	11
Binary Search	11
Interpolation Search	11
Jump and Ternary Search (Awareness)	11
Part C5 - Comparison-Based Sorting	13
Bubble Sort	13
Selection Sort	13
Insertion Sort	13
Merge Sort	13
Quick Sort	13
Heap Sort	13
Shell Sort	14

Comparison Table of Sorts	14
Lower Bound for Comparison Sorting	14
Part C6 - Non-Comparison Sorting	15
Counting Sort	15
Radix Sort	15
Bucket Sort	15
When Linear-Time Sorting Applies	15
Part C7 - Sorting Properties & Selection	16
Stability	16
In-Place vs Out-of-Place; Adaptive	16
Choosing a Sort by Input	16
kth Smallest / Largest (Order Statistics)	16
QuickSelect	16
Part D - Hashing	17
Hash Table and Direct Addressing	17
Hash Functions	17
Collision Resolution - Chaining	17
Open Addressing	17
Load Factor and Performance	17
Rehashing	17
Average vs Worst-Case Lookup	18
Part E9 - Divide and Conquer	19
General Paradigm	19
D&C; View of Classic Algorithms	19
Maximum Subarray	19
Strassen's Matrix Multiplication (Awareness)	19
Finding Min and Max Together	19
Part E10 - Greedy Algorithms	20
Greedy-Choice & Optimal Substructure	20
Activity Selection	20
Fractional Knapsack	20
Huffman Coding	20
Job Sequencing with Deadlines	20
Coin Change (Greedy)	20
MST and Dijkstra are Greedy	20

When Greedy Fails	20
Part E11 - Dynamic Programming	22
Overlapping Subproblems & Optimal Substructure	22
Memoization vs Tabulation	22
Classic DP Problems	22
DP on Grids	23
Floyd-Warshall as DP	23
Part E12 - Backtracking	24
State-Space Tree	24
N-Queens, Subsets, Permutations	24
Branch and Bound (Awareness)	24
Part F13 - Graph Basics	25
Representations	25
Directed/Undirected, Weighted/Unweighted	25
Part F14 - Graph Traversals	26
Breadth-First Search (BFS)	26
Depth-First Search (DFS)	26
Edge Types in DFS	26
Applications	26
Part F15 - Minimum Spanning Tree	28
Definition and Properties	28
Kruskal's Algorithm	28
Prim's Algorithm	28
Kruskal vs Prim	28
Union-Find (Disjoint Set Union)	28
Part F16 - Shortest Path	29
BFS for Unweighted Graphs	29
Dijkstra's Algorithm	29
Bellman-Ford	29
Floyd-Warshall	29
Path Reconstruction	29
Part G17 - Complexity Theory (Awareness)	30
P, NP, NP-Complete, NP-Hard	30
Examples of NP-Complete Problems	30

Decision vs Optimization	30
------------------------------------	----

Part H18 - CIL MT Exam Focus & Quick Revision	31
--	-----------

Identifying Complexity (Best/Avg/Worst)	31
Master Theorem Direct Application	31
Counting Comparisons/Swaps	31
Stability/In-Place Property	31
Tracing Dijkstra/Bellman-Ford/Floyd-Warshall	31
MST Construction Order	31
Greedy vs DP Applicability	31
Asymptotic Ordering	31
Hash Table Probe Sequence	31

Part A1 - Growth of Functions & Asymptotic Analysis

Why Asymptotic Analysis

- Asymptotic analysis measures how an algorithm's running time or space grows as the input size n grows toward infinity, ignoring machine-specific constants and lower-order terms.
- It makes analysis **machine-independent** and **input-representation-independent**: a fast computer only changes the hidden constant, not the growth class.
- We count the number of **basic/primitive operations** (comparisons, swaps, arithmetic) as a function of n , not wall-clock seconds.
- The dominant term decides the class: for $3n^2 + 5n + 7$ the growth is $O(n^2)$; constants **3, 5, 7** and the linear term are discarded.
- Asymptotic notation describes behaviour for **large n** ($n \rightarrow \text{infinity}$); for tiny inputs a "slower" algorithm may actually run faster due to small constants.

Memory Trick

- Think "**drop constants, keep the biggest term**": $2n^3 + 100n^2 \rightarrow O(n^3)$.

Big-O (Upper Bound)

- Big-O** gives an **asymptotic upper bound**: $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- It answers "the algorithm takes **at most** this much"; it is the **worst-case ceiling** on growth.
- Big-O is an upper bound only, so $n = O(n^2)$ is technically true (loose but valid).
- Used to state **worst-case** guarantees: binary search is $O(\log n)$, linear search is $O(n)$.

Big-Omega (Lower Bound)

- Big-Omega** gives an **asymptotic lower bound**: $f(n) = \Omega(g(n))$ if there exist $c > 0$, n_0 with $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
- It answers "the algorithm takes **at least** this much"; it is the **floor** on growth.
- Comparison-based sorting is $\Omega(n \log n)$ - no comparison sort can beat this lower bound.

Big-Theta (Tight Bound)

- Big-Theta** gives an **asymptotically tight bound**: $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ AND $f(n) = \Omega(g(n))$.
- Formally $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$ with $c_1, c_2 > 0$.
- Theta is the most precise statement: merge sort is $\Theta(n \log n)$ in all cases.
- If best and worst case differ (e.g. quicksort $\Theta(n \log n)$ avg, $\Theta(n^2)$ worst), there is no single Theta over all inputs.

Little-o and Little-omega

- Little-o**: $f(n) = o(g(n))$ means f grows **strictly slower** than g : the bound is **not tight** ($\lim f/g = 0$). Example $n = o(n^2)$.
- Little-omega**: $f(n) = \omega(g(n))$ means f grows **strictly faster** than g ($\lim f/g = \text{infinity}$). Example $n^2 = \omega(n)$.
- Mnemonic: **big-O** $\leq$, **little-o** $<$ (strict); **big-Omega** $\geq$, **little-omega** $>$ (strict); **Theta** $=$.

Ordering of Common Functions

- Standard increasing order of growth: $1 < \log \log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$.
- Classes by name: **constant** $O(1)$, **logarithmic** $O(\log n)$, **linear** $O(n)$, **linearithmic** $O(n \log n)$, **quadratic** $O(n^2)$, **cubic** $O(n^3)$, **exponential** $O(2^n)$, **factorial** $O(n!)$.
- Any **polynomial** beats any **logarithm**; any **exponential** beats any **polynomial**; any **factorial** beats any simple exponential.
- $\log n$** base does not matter asymptotically: **$\log 2 n$** and **$\log 10 n$** differ by a constant factor only.

Function	Class	Example algorithm
1	Constant	Array index access, hash lookup (avg)
$\log n$	Logarithmic	Binary search
n	Linear	Linear search, counting sort scan
$n \log n$	Linearithmic	Merge sort, heap sort
n^2	Quadratic	Bubble/selection/insertion sort
n^3	Cubic	Floyd-Warshall, naive matrix multiply
2^n	Exponential	Subset enumeration, naive TSP
$n!$	Factorial	Permutation generation, brute TSP

Comparing Growth Rates

- To compare $f(n)$ and $g(n)$, evaluate **$\lim_{n \rightarrow \infty} f(n)/g(n)$** : if **0** then $f = o(g)$; if **infinity** then $f = \omega(g)$; if a **positive constant** then $f = \Theta(g)$.
- Apply **logarithms** to compare hard cases: compare **$\log f$** vs **$\log g$** (e.g. **$n^{\log n}$** vs **$(\log n)^n$**).
- $n^{0.5}$** (i.e. $\sqrt{n}$) is between **$\log n$** and **n** .

Properties of Asymptotic Notation

- Transitivity**: if $f = O(g)$ and $g = O(h)$ then $f = O(h)$ (holds for O , Ω , Θ , o , ω).
- Reflexivity**: $f = O(f)$, $f = \Omega(f)$, $f = \Theta(f)$ (NOT true for little- o / ω).
- Symmetry**: $f = \Theta(g)$ iff $g = \Theta(f)$.
- Transpose symmetry**: $f = O(g)$ iff $g = \Omega(f)$; $f = o(g)$ iff $g = \omega(f)$.
- Sum rule**: $O(f) + O(g) = O(\max(f, g))$. **Product rule**: $O(f) \cdot O(g) = O(f \cdot g)$.

CBT Trap

- Big-O is an upper bound, NOT the worst case** - worst/avg/best are about inputs; $O/\Omega/\Theta$ are about bounds. You can have Big-O of the best case.
- $2^{(n+1)} = O(2^n)$** (constant factor 2) but **$2^{(2n)} = 4^n \neq O(2^n)$** .
- $\log(n!) = \Theta(n \log n)$** (Stirling) - a very common MCQ.
- $n \log n$** is NOT the same as **$\log(n^n)$** ... actually it IS: **$\log(n^n) = n \log n$** ; watch this identity.
- A function can be $O(g)$ and $\Omega(h)$ for different g, h ; only Θ pins it exactly.

Part A2 - Types of Analysis

Best, Average, Worst Case

- **Worst case**: maximum time over all inputs of size n - gives a **guarantee** (most used; e.g. quicksort worst $O(n^2)$).
- **Best case**: minimum time over all inputs - usually least useful (e.g. insertion sort best $O(n)$ on sorted data).
- **Average case**: expected time assuming a probability distribution of inputs (e.g. quicksort average $O(n \log n)$).
- For an MCQ, "complexity of X " with no qualifier almost always means **worst case**.

Algorithm	Best	Average	Worst
Linear search	$O(1)$	$O(n)$	$O(n)$
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Time vs Space Complexity

- **Time complexity** counts operations; **space complexity** counts extra memory beyond the input (**auxiliary space**).
- Merge sort: time $O(n \log n)$, auxiliary space $O(n)$; quicksort: time avg $O(n \log n)$, space $O(\log n)$ for recursion stack.
- **In-place** algorithms use $O(1)$ or $O(\log n)$ auxiliary space (heap sort, quicksort, bubble/selection/insertion).
- **Time-space tradeoff**: hashing/memoization spend memory to save time.

Counting Basic Operations

- Count the most frequent **dominant operation**: comparisons in searching/sorting, multiplications in matrix code.
- A single loop $1..n \rightarrow O(n)$; nested loops both to $n \rightarrow O(n^2)$; loop that halves n each step $\rightarrow O(\log n)$.
- Loop $i = 1; i < n; i = i * 2$ runs $\log_2 n$ times; loop $i = 1; i < n; i = i * 3$ runs $\log_3 n$ times.

Amortized Analysis (Awareness)

- **Amortized analysis** averages the cost of a sequence of operations so an occasional expensive operation is spread over many cheap ones.
- **Dynamic array** (doubling) push is $O(1)$ amortized though a single resize is $O(n)$.
- Three methods: **aggregate**, **accounting (banker's)**, and **potential** methods.

CBT Trap

- Average case **!=** amortized: average assumes input distribution; amortized is a **worst-case guarantee over a sequence** with no probability.
- Insertion sort worst $O(n^2)$ occurs on **reverse-sorted** input, best $O(n)$ on **already-sorted**.

Part A3 - Recurrence Relations

Forming a Recurrence

- A **recurrence** expresses $T(n)$ in terms of T of smaller inputs plus the non-recursive work.
- A function that recurses once on $n-1$ and does $O(1)$ work gives $T(n) = T(n-1) + c \rightarrow O(n)$.
- One recursion on $n/2$ plus $O(1)$ gives $T(n) = T(n/2) + c \rightarrow O(\log n)$ (binary search).
- Two recursions on $n/2$ plus $O(n)$ merge gives $T(n) = 2T(n/2) + n \rightarrow O(n \log n)$ (merge sort).

Substitution Method

- Substitution method**: guess the form of the solution, then prove it by **mathematical induction**, finding constants.
- Works best when you already suspect the bound (e.g. guess $O(n \log n)$ and verify).

Recursion Tree Method

- Recursion tree**: draw work done at each level, sum across levels, multiply by number of nodes per level.
- For $T(n) = 2T(n/2) + n$: each level does n work, there are $\log n + 1$ levels $\rightarrow$ total $n \log n = O(n \log n)$.
- For $T(n) = 2T(n/2) + n^2$: work per level **decreases geometrically**, dominated by root $n^2 \rightarrow O(n^2)$.

Master Theorem

- The **Master Theorem** solves recurrences of form $T(n) = aT(n/b) + f(n)$ with $a \geq 1, b > 1$.
- Compute the critical exponent $c_{\text{crit}} = \log_b(a)$ and compare $f(n)$ with $n^{c_{\text{crit}}}$.
- Case 1**: if $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$ (leaves dominate).
- Case 2**: if $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$ (balanced).
- Case 3**: if $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity holds $\rightarrow T(n) = \Theta(f(n))$ (root dominates).

Recurrence	a, b, f(n)	$\log_b a$	Result
$T(n) = 2T(n/2) + n$	2, 2, n	1	$\Theta(n \log n)$ (case 2)
$T(n) = 2T(n/2) + 1$	2, 2, 1	1	$\Theta(n)$ (case 1)
$T(n) = 4T(n/2) + n$	4, 2, n	2	$\Theta(n^2)$ (case 1)
$T(n) = T(n/2) + 1$	1, 2, 1	0	$\Theta(\log n)$ (case 2)
$T(n) = 3T(n/2) + n$	3, 2, n	$\log_2 3 \sim 1.58$	$\Theta(n^{1.58})$ (case 1)
$T(n) = 2T(n/2) + n^2$	2, 2, n^2	1	$\Theta(n^2)$ (case 3)

Common Recurrences Cheat Sheet

Recurrence	Solution	Source
$T(n) = T(n-1) + c$	$O(n)$	Linear recursion, factorial
$T(n) = T(n-1) + n$	$O(n^2)$	Insertion/selection sort
$T(n) = 2T(n-1) + c$	$O(2^n)$	Tower of Hanoi
$T(n) = T(n/2) + c$	$O(\log n)$	Binary search
$T(n) = 2T(n/2) + c$	$O(n)$	Tree traversal
$T(n) = 2T(n/2) + n$	$O(n \log n)$	Merge sort

CBT Trap

- Master theorem does NOT apply when $a < 1$, $b \leq 1$, $f(n)$ is **not polynomial**, or the gap between f and $n^{\log_b a}$ is **not polynomial** (e.g. $f = n \log n$ with $\log_b a = 1$ falls in the gap).
- **Tower of Hanoi** needs $2^n - 1$ moves $\rightarrow T(n) = 2T(n-1) + 1 \rightarrow O(2^n)$; do not confuse with $n/2$ recurrences.
- **$\log_b a$** uses base b (the divisor), not base 2 always.

Part B - Searching

Linear / Sequential Search

- **Linear search** scans elements one by one until the key is found or the list ends.
- Works on **unsorted** data; no precondition required.
- Best case **$O(1)$** (first element), average **$O(n/2)=O(n)$** , worst **$O(n)$** ; space **$O(1)$** .
- Number of comparisons in worst case = **n** ; average successful = **$(n+1)/2$** .

Binary Search

- **Binary search** repeatedly halves a **sorted** array, comparing the key to the middle element.
- **Precondition**: array MUST be **sorted**; otherwise result is undefined.
- Mid computation: use **$mid = low + (high - low)/2$** to avoid integer **overflow** (not **$(low+high)/2$**).
- Complexity: best **$O(1)$** , average/worst **$O(\log_2 n)$** ; space **$O(1)$** iterative, **$O(\log n)$** recursive (stack).
- Maximum comparisons = **$\text{floor}(\log_2 n) + 1$** ; recurrence **$T(n)=T(n/2)+O(1)$** .

Iterative vs Recursive

- **Iterative** binary search uses a loop and **$O(1)$** space.
- **Recursive** binary search calls itself on half, using **$O(\log n)$** stack space.

First / Last Occurrence Variants

- To find **first occurrence** of a duplicate key, on a match keep searching the **left half**; for **last occurrence**, search the **right half**.
- Used to count occurrences = **$last - first + 1$** in **$O(\log n)$** .

Interpolation Search

- **Interpolation search** estimates the probe position using value distribution: **$pos = low + ((key - a[low]) * (high - low)) / (a[high] - a[low])$** .
- Best for **uniformly distributed sorted** data; average **$O(\log \log n)$** , worst **$O(n)$** (skewed data).
- Analogous to how humans look up a name in a phone book.

Jump and Ternary Search (Awareness)

- **Jump search** on sorted data jumps in blocks of **$\sqrt{n}$** then linear-scans the block; complexity **$O(\sqrt{n})$** .
- **Ternary search** splits into **3 parts** (two mid points); complexity **$O(\log_3 n)$** comparisons but more comparisons per step than binary, so binary is usually faster.

Search	Precondition	Best	Average	Worst
Linear	none	$O(1)$	$O(n)$	$O(n)$
Binary	sorted	$O(1)$	$O(\log n)$	$O(\log n)$
Interpolation	sorted, uniform	$O(1)$	$O(\log \log n)$	$O(n)$
Jump	sorted	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
Ternary	sorted	$O(1)$	$O(\log_3 n)$	$O(\log_3 n)$

CBT Trap

- Binary search needs **sorted** input; if asked "search unsorted array fastest", answer is **linear search** (sorting first costs $O(n \log n)$).
- **$mid = (low+high)/2$** can **overflow**; the safe form is **$low + (high-low)/2$** .
- Ternary search makes **more** comparisons than binary search despite fewer levels - binary search is optimal among comparison searches.

Part C5 - Comparison-Based Sorting

Bubble Sort

- **Bubble sort** repeatedly compares **adjacent** elements and swaps them if out of order; largest "bubbles" to the end each pass.
- After pass k , the largest k elements are in final position; needs up to $n-1$ passes.
- Complexity: best $O(n)$ with early-exit on sorted input, average/worst $O(n^2)$; space $O(1)$.
- Comparisons worst = $n(n-1)/2$; swaps worst = $n(n-1)/2$.
- **Early-exit optimization**: if a pass makes **0 swaps**, the array is sorted $\rightarrow$ stop (gives $O(n)$ best case).
- **Stable** and **in-place**; adaptive with the flag optimization.

Selection Sort

- **Selection sort** repeatedly selects the **minimum** of the unsorted part and places it at the front.
- Complexity: best = average = worst = $O(n^2)$ (always scans the rest); space $O(1)$.
- Comparisons always = $n(n-1)/2$; swaps = at most $n-1$ (fewest swaps of the simple sorts).
- **NOT stable** (standard version) and **not adaptive**; it is **in-place**.

Insertion Sort

- **Insertion sort** builds a sorted prefix, inserting each new element into its correct position by shifting larger elements right.
- Complexity: best $O(n)$ (already sorted), average/worst $O(n^2)$ (reverse sorted); space $O(1)$.
- **Stable**, **in-place**, **adaptive**, and **online** (can sort as data arrives).
- Best choice for **small** or **nearly-sorted** arrays; used as base case in hybrid sorts (Timsort, Introsort).

Merge Sort

- **Merge sort** is **divide-and-conquer**: split array in half, recursively sort, then **merge** two sorted halves.
- The **merge step** compares fronts of two halves and is $O(n)$ per level.
- Complexity: best = average = worst = $O(n \log n)$ (guaranteed); recurrence $T(n)=2T(n/2)+n$.
- Auxiliary space $O(n)$ (out-of-place); **stable**; **not in-place**.
- Preferred for **linked lists** ($O(1)$ extra space there) and **external sorting** of huge files.

Quick Sort

- **Quick sort** picks a **pivot**, **partitions** elements into those $<$ pivot and $>$ pivot, then recursively sorts each side.
- **Lomuto partition**: pivot = last element, single index scan, simpler, more swaps.
- **Hoare partition**: two pointers from both ends, fewer swaps, original Hoare scheme.
- Complexity: best/average $O(n \log n)$, worst $O(n^2)$ (already-sorted with bad pivot); space $O(\log n)$ avg recursion.
- **In-place**, **NOT stable**; fastest in practice due to cache locality and low constants.
- Worst case avoided via **randomized pivot** or **median-of-three** pivot selection.

Heap Sort

- **Heap sort** builds a **max-heap** then repeatedly extracts the max to the end.

- **Build-heap** is $O(n)$; each of n extractions costs $O(\log n)$ -> total $O(n \log n)$.
- Complexity: best = average = worst = $O(n \log n)$; space $O(1)$ (in-place).
- **NOT stable**; no extra memory; good worst-case guarantee but poor cache locality (slower than quicksort in practice).

Shell Sort

- **Shell sort** is a **generalized insertion sort** using decreasing **gap** sequences; far apart elements are compared first.
- Complexity depends on gap sequence: worst $O(n^2)$ (Shell's original), $O(n^{1.5})$ or $O(n \log^2 n)$ for better gaps.
- **In-place**, **NOT stable**, adaptive.

Comparison Table of Sorts

Sort	Best	Average	Worst	Space	Stable	In-place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Shell	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	No	Yes

Lower Bound for Comparison Sorting

- Any **comparison-based** sort must make $\Omega(n \log n)$ comparisons in the worst case.
- Proof idea: a decision tree with $n!$ leaves has height $\geq \log_2(n!) = \Theta(n \log n)$.

CBT Trap

- **Stable sorts**: Bubble, Insertion, Merge, Counting, Radix. **Unstable**: Selection, Quick, Heap, Shell. Mnemonic for stable: "**BIM-CR**".
- **Selection sort always does $n(n-1)/2$ comparisons** regardless of input - it is NOT adaptive.
- **Quicksort worst case $O(n^2)$** happens on **sorted/reverse-sorted** input with first/last pivot - randomization fixes it.
- **Heap sort** is the in-place $O(n \log n)$ worst-case sort; merge sort is $O(n \log n)$ but **not in-place**.
- **Number of swaps**: selection sort = $O(n)$, bubble sort = $O(n^2)$ - selection minimizes writes.
- Build-heap is $O(n)$, not $O(n \log n)$ - a classic trick question.

Part C6 - Non-Comparison Sorting

Counting Sort

- **Counting sort** counts occurrences of each key in a known range $[0, k]$, then computes positions from prefix sums.
- Complexity: $O(n + k)$ time, $O(n + k)$ space, where k is the range of input values.
- **Stable** (when built with prefix sums right-to-left); **not in-place**.
- Efficient only when $k = O(n)$; if k is huge it wastes memory.

Radix Sort

- **Radix sort** sorts numbers digit by digit using a stable sort (usually counting sort) per digit.
- **LSD radix** (least significant digit first) processes from rightmost digit; **MSD** from leftmost.
- Complexity: $O(d(n + b))$ where d = number of digits, b = base/radix.
- The per-digit sort **MUST be stable** for radix sort to work correctly.

Bucket Sort

- **Bucket sort** distributes elements into **buckets** over the value range, sorts each bucket, then concatenates.
- Best/average $O(n + k)$ (or $O(n)$) for uniformly distributed input; worst $O(n^2)$ if all fall in one bucket.
- Assumes input is **uniformly distributed** over a range; commonly sorts floats in $[0, 1)$.

When Linear-Time Sorting Applies

- Linear-time sorts beat the $\Omega(n \log n)$ bound because they are **not comparison-based**.
- Applicable when keys are **integers in a small range** (counting), **fixed-width integers/strings** (radix), or **uniformly distributed** (bucket).

Sort	Time	Space	Stable	Assumption
Counting	$O(n+k)$	$O(n+k)$	Yes	keys in $[0, k]$
Radix (LSD)	$O(d(n+b))$	$O(n+b)$	Yes	fixed digits
Bucket	$O(n+k)$ avg	$O(n+k)$	Yes	uniform distribution

CBT Trap

- These do NOT violate the $\Omega(n \log n)$ lower bound because that bound applies ONLY to **comparison sorts**.
- Radix sort needs a **stable** subroutine; counting sort is the usual choice.
- Counting sort range k appears in complexity - if $k \gg n$ it is worse than merge sort.

Part C7 - Sorting Properties & Selection

Stability

- A sort is **stable** if it preserves the **relative order** of equal keys.
- Stable: **Bubble, Insertion, Merge, Counting, Radix, Bucket**; Unstable: **Selection, Quick, Heap, Shell**.
- Stability matters when sorting by **multiple keys** (e.g. sort by name then by date keeping name order).

In-Place vs Out-of-Place; Adaptive

- **In-place**: uses **$O(1)$** or **$O(\log n)$** extra space (Quick, Heap, Insertion, Selection, Bubble, Shell).
- **Out-of-place**: needs **$O(n)$** extra (Merge, Counting, Radix, Bucket).
- **Adaptive**: runs faster on **nearly sorted** input (Insertion, Bubble with flag); Selection is **non-adaptive**.

Choosing a Sort by Input

- **Small array** -> insertion sort (low constants).
- **Nearly sorted** -> insertion or bubble (adaptive, near $O(n)$).
- **Worst-case guarantee + in-place** -> heap sort.
- **Stability required** -> merge sort.
- **General fastest average** -> quick sort (with randomized pivot).
- **Small integer keys** -> counting/radix sort.
- **Linked list** -> merge sort.

kth Smallest / Largest (Order Statistics)

- The **kth order statistic** is the kth smallest element; the **median** is the $n/2$ -th.
- Naive: sort then index -> **$O(n \log n)$** ; heap of size k -> **$O(n \log k)$** .
- **Median of medians** gives guaranteed **$O(n)$** worst-case selection.

QuickSelect

- **QuickSelect** uses quicksort's partition but recurses into only **one side** (the side containing rank k).
- Expected **$O(n)$** , worst **$O(n^2)$** ; space **$O(1)/O(\log n)$** .
- Used to find median or kth element without full sorting.

CBT Trap

- QuickSelect average is **$O(n)$** , NOT $O(n \log n)$ - recursing on one side gives **$n + n/2 + n/4 + \dots = 2n$** .
- Selection sort being unstable surprises many; the **stable** simple sorts are bubble and insertion.
- Finding **both min and max** needs only **$3 \cdot \text{floor}(n/2)$** comparisons (pairwise), not $2n-2$.

Part D - Hashing

Hash Table and Direct Addressing

- A **hash table** maps keys to array indices via a **hash function** $h(k)$, giving average **$O(1)$** insert/search/delete.
- **Direct addressing** uses the key itself as index - works only for **small integer key universes**.
- **Collision**: two distinct keys hash to the same slot; resolution is the core problem of hashing.

Hash Functions

- **Division method**: $h(k) = k \bmod m$; choose m a **prime** not close to a power of 2.
- **Multiplication method**: $h(k) = \text{floor}(m * (k * A \bmod 1))$ with $0 < A < 1$ (Knuth suggests $A \sim 0.6180339$ the golden ratio).
- A good hash function is **deterministic**, **uniform** (spreads keys), and **fast**.

Collision Resolution - Chaining

- **Separate chaining** stores all keys hashing to a slot in a **linked list** (or list/tree) at that slot.
- **Load factor $\alpha = n/m$** (entries / slots); average search = **$O(1 + \alpha)$** .
- Chaining handles **$\alpha > 1$** gracefully; worst case all in one chain $\rightarrow O(n)$.

Open Addressing

- **Open addressing** stores all entries in the table itself; on collision it **probes** for the next free slot. Requires **$\alpha \leq 1$** .

Linear Probing

- **Linear probing**: $h(k, i) = (h'(k) + i) \bmod m$; probe slots one by one.
- Suffers **primary clustering**: long runs of occupied slots form and grow.

Quadratic Probing

- **Quadratic probing**: $h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$.
- Suffers **secondary clustering** (keys with same initial probe follow same sequence) but avoids primary clustering.

Double Hashing

- **Double hashing**: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$; the step depends on a second hash.
- Best probe distribution of the three; **$h_2(k)$** must be **relatively prime** to m to cover all slots.

Load Factor and Performance

- **Load factor $\alpha = n/m$** ; performance degrades as $\alpha \rightarrow 1$ for open addressing.
- Open-addressing expected probes (successful) $\sim (1/2)(1 + 1/(1-\alpha))$; unsuccessful $\sim 1/(1-\alpha)$.
- Typical resize/rehash threshold is **$\alpha = 0.75$** (e.g. Java HashMap default).

Rehashing

- **Rehashing**: when load factor exceeds a threshold, allocate a **larger table** (often **double**) and reinsert all keys.

- Rehash cost $O(n)$ but **amortized $O(1)$** per insertion.

Average vs Worst-Case Lookup

- Average lookup **$O(1)$** with a good hash and low load factor.
- Worst case **$O(n)$** when all keys collide (degenerate to a list).

Method	Extra structure	Clustering	Load factor
Chaining	linked lists	none	can exceed 1
Linear probing	none (in-table)	primary	≤ 1
Quadratic probing	none	secondary	≤ 1
Double hashing	none	minimal	≤ 1

CBT Trap

- **Linear probing** -> **primary clustering**; **quadratic probing** -> **secondary clustering**; **double hashing** minimizes both - know which is which.
- For division method choose m **prime**, away from powers of 2; powers of 2 use only low-order bits.
- Chaining can hold **$\alpha > 1$** ; open addressing CANNOT (table full at $\alpha = 1$).
- Deletion in open addressing needs a **tombstone/deleted marker**, not a plain empty slot.

Part E9 - Divide and Conquer

General Paradigm

- **Divide and conquer** has three steps: **Divide** the problem into subproblems, **Conquer** (solve recursively), **Combine** the results.
- Typical recurrence $T(n) = aT(n/b) + f(n)$ solved by the Master Theorem.

D&C View of Classic Algorithms

- **Binary search**: divide in 2, recurse on one half, no combine $\rightarrow O(\log n)$.
- **Merge sort**: divide in 2, recurse both, combine = merge $\rightarrow O(n \log n)$.
- **Quick sort**: divide = partition, recurse both, no explicit combine $\rightarrow$ avg $O(n \log n)$.

Maximum Subarray

- **Maximum subarray** (max contiguous sum) by D&C: best crossing the mid + best in left + best in right $\rightarrow O(n \log n)$.
- **Kadane's algorithm** solves it in $O(n)$ via dynamic programming (better than D&C).

Strassen's Matrix Multiplication (Awareness)

- Naive matrix multiply is $O(n^3)$; **Strassen's** uses **7** multiplications instead of 8 per recursion.
- Strassen complexity = $O(n^{\log_2 7}) \sim O(n^{2.807})$.

Finding Min and Max Together

- Scanning separately costs $2n - 2$ comparisons; pairwise (D&C) costs $3 \cdot \text{floor}(n/2) \sim 1.5n$ comparisons.

CBT Trap

- Strassen exponent is $\log_2 7 \sim 2.807$, NOT 2.81 from 7 muls of 8 - memorize $n^{2.807}$.
- Maximum subarray optimal is **Kadane** $O(n)$, not the D&C $O(n \log n)$.

Part E10 - Greedy Algorithms

Greedy-Choice & Optimal Substructure

- A **greedy algorithm** makes the **locally optimal** choice at each step hoping for a global optimum.
- Works when the problem has the **greedy-choice property** (a global optimum contains a local greedy choice) and **optimal substructure**.

Activity Selection

- **Activity selection**: pick maximum non-overlapping activities by always choosing the one that **finishes earliest**.
- Sort by **finish time**, then greedily select; **$O(n \log n)$** (sorting dominated).

Fractional Knapsack

- **Fractional knapsack**: take items by highest **value/weight ratio**, taking fractions allowed -> greedy is **optimal**.
- Complexity **$O(n \log n)$** for the sort.

Huffman Coding

- **Huffman coding** builds an optimal **prefix code** by repeatedly merging the **two lowest-frequency** nodes using a min-heap.
- Produces a **prefix-free** binary tree; more frequent symbols get **shorter** codes.
- Complexity **$O(n \log n)$** ; it is a **greedy** algorithm and gives the **optimal** prefix code.

Job Sequencing with Deadlines

- **Job sequencing**: each job has a **deadline** and **profit**; schedule to maximize profit, one job per slot.
- Greedy: sort by **profit descending**, assign each job to the latest free slot before its deadline; **$O(n^2)$** or **$O(n \log n)$** with union-find.

Coin Change (Greedy)

- **Greedy coin change** works for **canonical** coin systems (e.g. Indian/US coins) - always pick the largest coin $\leq$ remaining.
- **Fails** for systems like **[1, 3, 4]** to make **6** (greedy gives $4+1+1=3$ coins; optimal is $3+3=2$ coins).

MST and Dijkstra are Greedy

- **Kruskal**, **Prim**, and **Dijkstra** are all **greedy** algorithms (cross-reference graph section).

When Greedy Fails

- Greedy **fails** on **0/1 knapsack** (items indivisible) - needs **dynamic programming**.

CBT Trap

- **Fractional knapsack** = greedy (optimal); **0/1 knapsack** = DP (greedy fails) - the single most common greedy-vs-DP MCQ.
- Activity selection sorts by **finish time**, NOT start time or duration.
- Huffman merges the **two smallest** frequencies each step.

Part E11 - Dynamic Programming

Overlapping Subproblems & Optimal Substructure

- **Dynamic programming** applies when a problem has **overlapping subproblems** (same subproblem solved repeatedly) AND **optimal substructure**.
- DP **stores** subproblem results to avoid recomputation (vs plain recursion which recomputes).

Memoization vs Tabulation

- **Memoization** (top-down): recursion + a cache; computes only needed subproblems, uses **recursion stack**.
- **Tabulation** (bottom-up): iterative, fills a table from base cases up; usually **faster**, no recursion overhead.

Aspect	Memoization	Tabulation
Direction	Top-down	Bottom-up
Style	Recursive	Iterative
Computes	only needed	all subproblems
Overhead	recursion stack	none

Classic DP Problems

Fibonacci / Climbing Stairs

- **Fibonacci** via DP: $F(n) = F(n-1) + F(n-2)$; DP makes it $O(n)$ time vs $O(2^n)$ naive recursion.
- **Climbing stairs** (1 or 2 steps) has the same recurrence as Fibonacci.

0/1 Knapsack

- **0/1 knapsack**: maximize value with weight $\leq W$, each item taken **0 or 1** time.
- DP table $dp[i][w]$; complexity $O(nW)$ (pseudo-polynomial) - depends on capacity W .

Longest Common Subsequence (LCS)

- **LCS**: longest subsequence common to two strings; recurrence: if chars match $dp[i][j] = dp[i-1][j-1] + 1$ else $\max(dp[i-1][j], dp[i][j-1])$.
- Complexity $O(m.n)$ time and space.

Longest Increasing Subsequence (LIS)

- **LIS**: longest strictly increasing subsequence; DP $O(n^2)$, or $O(n \log n)$ using binary search + patience.

Matrix Chain Multiplication

- **Matrix chain multiplication**: parenthesize a product to minimize scalar multiplications; DP $O(n^3)$ time, $O(n^2)$ space.
- It is an **optimization over parenthesizations**, not actually multiplying matrices.

Edit Distance

- **Edit distance** (Levenshtein): minimum **insert/delete/replace** operations to convert one string to another; DP $O(m.n)$.

Coin Change (DP)

- **Coin change - count ways**: number of ways to form amount; **coin change - min coins**: fewest coins.
- DP complexity **$O(\text{amount} * \text{coins})$** ; handles non-canonical systems where greedy fails.

Subset Sum

- **Subset sum**: does a subset sum to target T? DP **$O(n.T)$** (pseudo-polynomial); it is **NP-complete** in general.

Rod Cutting

- **Rod cutting**: cut a rod to maximize revenue from piece prices; DP **$O(n^2)$** ; recurrence **$r(n) = \max(p[i] + r(n-i))$** .

DP on Grids

- **Minimum path sum** / unique paths on a grid: **$dp[i][j] = \text{cost} + \min(\text{top}, \text{left})$** ; **$O(m.n)$** .

Floyd-Warshall as DP

- **Floyd-Warshall** is DP over intermediate vertices; all-pairs shortest path **$O(V^3)$** (cross-ref shortest path).

Problem	Time	Space
Fibonacci	$O(n)$	$O(1)$ optimized
0/1 Knapsack	$O(nW)$	$O(nW)$
LCS	$O(mn)$	$O(mn)$
LIS	$O(n \log n)$	$O(n)$
Matrix chain	$O(n^3)$	$O(n^2)$
Edit distance	$O(mn)$	$O(mn)$
Subset sum	$O(nT)$	$O(nT)$

CBT Trap

- **0/1 knapsack is $O(nW)$ "pseudo-polynomial"** - polynomial in the numeric value W, not its bit-length; this is why it is NP-hard yet has a DP.
- **LIS in $O(n \log n)$** uses binary search, not pure DP $O(n^2)$.
- DP needs BOTH **overlapping subproblems** and **optimal substructure**; divide-and-conquer has optimal substructure but **independent** (non-overlapping) subproblems.
- **Matrix chain** minimizes multiplications; it does not perform the multiplication.

Part E12 - Backtracking

State-Space Tree

- **Backtracking** explores a **state-space tree** via DFS, abandoning a path (**pruning/bounding**) as soon as it cannot lead to a solution.
- It is a **systematic brute force** with pruning.

N-Queens, Subsets, Permutations

- **N-Queens**: place n queens on $n \times n$ board so none attack; backtracking tries column by column, pruning on same row/diagonal.
- **Subsets**: generate all 2^n subsets; **permutations**: generate all $n!$ orderings.

Branch and Bound (Awareness)

- **Branch and bound** extends backtracking for **optimization**: it computes **bounds** to prune branches that cannot beat the best solution found.
- Used for **0/1 knapsack**, **TSP** optimization.

CBT Trap

- **Backtracking** is for **feasibility/all-solutions** via DFS; **branch and bound** is for **optimization** using bounds.
- Number of subsets = 2^n ; permutations = $n!$ - memorize both.

Part F13 - Graph Basics

Representations

- **Adjacency matrix**: $V \times V$ 2D array; edge lookup $O(1)$; space $O(V^2)$; good for **dense** graphs.
- **Adjacency list**: array of lists; space $O(V+E)$; edge lookup $O(\text{degree})$; good for **sparse** graphs.
- Iterating all neighbors: matrix $O(V)$ per vertex, list $O(\text{degree})$.

Operation	Adjacency matrix	Adjacency list
Space	$O(V^2)$	$O(V+E)$
Edge exists?	$O(1)$	$O(\text{degree})$
Iterate neighbors	$O(V)$	$O(\text{degree})$
Add edge	$O(1)$	$O(1)$
Best for	dense	sparse

Directed/Undirected, Weighted/Unweighted

- **Undirected** edge $\{u,v\}$ appears **twice** in adjacency list; sum of degrees = $2E$ (handshaking lemma).
- **Directed** graph distinguishes in-degree and out-degree; sum of out-degrees = E .
- **Weighted** graphs store weight with each edge; unweighted use unit cost.

CBT Trap

- Adjacency matrix space $O(V^2)$ even for few edges - wasteful for **sparse** graphs; use lists.
- Sum of all degrees in undirected graph = $2E$ (**handshaking lemma**); number of odd-degree vertices is always **even**.

Part F14 - Graph Traversals

Breadth-First Search (BFS)

- **BFS** explores level by level using a **FIFO queue** and a **visited** array.
- Complexity **$O(V+E)$** with adjacency list; **$O(V^2)$** with matrix.
- BFS finds **shortest path** (fewest edges) in **unweighted** graphs.
- Visits nodes in order of **increasing distance** from source.

Depth-First Search (DFS)

- **DFS** explores as deep as possible before backtracking, using **recursion** or an explicit **stack**.
- Complexity **$O(V+E)$** list, **$O(V^2)$** matrix.
- Produces a **DFS tree/forest** with **discovery** and **finish** times.

Edge Types in DFS

- DFS classifies edges as **tree**, **back**, **forward**, and **cross** edges.
- A **back edge** indicates a **cycle**; undirected DFS has only **tree** and **back** edges.

Applications

Connected Components

- **Connected components**: run DFS/BFS from each unvisited node; each run finds one component; **$O(V+E)$** .

Cycle Detection

- **Undirected cycle**: a **back edge** to a non-parent visited node; or union-find detects cycle on edge within same set.
- **Directed cycle**: a **back edge** to a node currently in the **recursion stack** (gray node).

Bipartite Check (2-Coloring)

- **Bipartite check**: 2-color the graph via BFS/DFS; a graph is bipartite iff it has **no odd-length cycle**.

Topological Sort

- **Topological sort**: linear order of a **DAG** where every edge $u \rightarrow v$ has u before v .
- **DFS-based**: push nodes on finish, reverse the order.
- **Kahn's algorithm** (BFS-based): repeatedly remove nodes of **in-degree 0**; both **$O(V+E)$** .
- Only **DAGs** (no cycles) have a topological order.

Strongly Connected Components (Awareness)

- **SCC**: maximal sets where every vertex reaches every other; found by **Kosaraju** (2 DFS) or **Tarjan** (1 DFS) in **$O(V+E)$** .

Traversal	Data structure	Shortest path?	Time
BFS	Queue (FIFO)	Yes (unweighted)	$O(V+E)$
DFS	Stack/recursion	No	$O(V+E)$

CBT Trap

- **BFS uses a queue, DFS uses a stack** - the single most common traversal MCQ.
- BFS gives shortest paths **only in unweighted** graphs; for weighted use Dijkstra.
- Directed cycle detection uses the **recursion-stack (gray)** set, NOT just visited.
- Topological sort exists **only for DAGs**; a cycle means no valid order.

Part F15 - Minimum Spanning Tree

Definition and Properties

- A **spanning tree** connects all V vertices with **$V-1$** edges and no cycle.
- A **minimum spanning tree (MST)** is a spanning tree of minimum total edge weight.
- MST is unique if all edge weights are **distinct**; **cut property**: the minimum-weight edge crossing any cut is in some MST.

Kruskal's Algorithm

- **Kruskal's**: sort all edges ascending, add the next edge if it does not form a **cycle** (checked via **union-find**).
- Complexity **$O(E \log E) = O(E \log V)$** (dominated by sorting).
- Builds the MST as a **forest** that merges; great for **sparse** graphs.

Prim's Algorithm

- **Prim's**: grow the MST from a start vertex, repeatedly adding the **minimum-weight edge** leaving the tree.
- **Array/matrix** version **$O(V^2)$** ; **binary heap** version **$O(E \log V)$** ; **Fibonacci heap** **$O(E + V \log V)$** .
- Great for **dense** graphs (matrix version).

Kruskal vs Prim

Aspect	Kruskal	Prim
Approach	edge-based, sort edges	vertex-based, grow tree
Data structure	union-find	priority queue
Complexity	$O(E \log E)$	$O(E \log V)$ or $O(V^2)$
Best for	sparse	dense

Union-Find (Disjoint Set Union)

- **Union-Find** supports **make-set**, **find** (which set), **union** (merge sets).
- **Path compression** + **union by rank** give near-constant **$O(\alpha(n))$** (inverse Ackermann) per operation.
- Used by Kruskal for cycle detection.

CBT Trap

- A spanning tree of V vertices has exactly **$V-1$** edges.
- **Kruskal = sort edges + union-find**; **Prim = priority queue grow** - do not swap them.
- Prim with **adjacency matrix** is **$O(V^2)$** (best for dense); with heap **$O(E \log V)$** .
- MST is unique only when all edge weights are **distinct**.

Part F16 - Shortest Path

BFS for Unweighted Graphs

- **BFS** computes single-source shortest paths in **unweighted** graphs in **$O(V+E)$** .

Dijkstra's Algorithm

- **Dijkstra**: single-source shortest path for **non-negative** edge weights; greedy, finalizes the closest unvisited vertex each step.
- **Array** version **$O(V^2)$** ; **binary heap** **$O((V+E) \log V)$** ; **Fibonacci heap** **$O(E + V \log V)$** .
- **FAILS** with **negative edge weights** (a finalized vertex may later get a shorter path).

Bellman-Ford

- **Bellman-Ford**: single-source shortest path that handles **negative weights**; relaxes all edges **$V-1$** times.
- Complexity **$O(VE)$** .
- A V -th relaxation that still improves signals a **negative-weight cycle**.

Floyd-Warshall

- **Floyd-Warshall**: **all-pairs** shortest path via DP over intermediate vertices; **$O(V^3)$** time, **$O(V^2)$** space.
- Handles **negative edges** (no negative cycles); detects negative cycle if any **$\text{dist}[i][i] < 0$** .

Path Reconstruction

- Store a **predecessor/parent** array during relaxation; backtrack from destination to source to rebuild the path.

Algorithm	Scope	Neg weights	Complexity
BFS	single-source unweighted	n/a	$O(V+E)$
Dijkstra	single-source	No	$O((V+E) \log V)$
Bellman-Ford	single-source	Yes	$O(VE)$
Floyd-Warshall	all-pairs	Yes	$O(V^3)$

CBT Trap

- **Dijkstra fails on negative edges**; use **Bellman-Ford** for negatives - core MCQ.
- **Bellman-Ford relaxes $V-1$ times**; the V -th pass detects **negative cycles**.
- **Floyd-Warshall is $O(V^3)$** all-pairs DP; Dijkstra from every source is **$O(V * (E \log V))$** .
- Dijkstra is **greedy**; Bellman-Ford and Floyd-Warshall are **dynamic programming**.

Part G17 - Complexity Theory (Awareness)

P, NP, NP-Complete, NP-Hard

- **P**: problems solvable in **polynomial time** by a deterministic machine.
- **NP**: problems whose solutions are **verifiable** in polynomial time (nondeterministic polynomial).
- **NP-Complete**: the hardest problems IN NP; every NP problem reduces to them; if one is in P then **P = NP**.
- **NP-Hard**: at least as hard as NP-complete, but **not necessarily in NP** (may not be decidable/verifiable).
- Known relationship: **P subset of NP**; whether **P = NP** is the famous **open problem**.

Examples of NP-Complete Problems

- Classic NP-complete: **SAT** (first proven, Cook-Levin), **3-SAT**, **TSP (decision)**, **vertex cover**, **clique**, **Hamiltonian cycle**, **subset sum**, **graph coloring**.

Decision vs Optimization

- A **decision problem** has a **yes/no** answer; an **optimization problem** seeks the best value.
- Complexity classes (NP) are defined over **decision** problems.

Class	Solve in P?	Verify in P?	In NP?
P	Yes	Yes	Yes
NP	unknown	Yes	Yes
NP-Complete	unknown	Yes	Yes
NP-Hard	unknown	not necessarily	not necessarily

CBT Trap

- **NP does NOT mean "non-polynomial"**; it means **nondeterministic polynomial / verifiable in poly time**.
- **NP-Complete = in NP AND NP-Hard**; NP-Hard need not be in NP.
- **SAT** was the **first** proven NP-complete (**Cook-Levin theorem**).
- **P subset of NP** is known; **P = NP** is **unproven**.

Part H18 - CIL MT Exam Focus & Quick Revision

Identifying Complexity (Best/Avg/Worst)

- Memorize the full sort table: **quicksort worst $O(n^2)$, merge/heap always $O(n \log n)$.**
- Single halving loop $\rightarrow O(\log n)$; nested halving $\rightarrow O(\log^2 n)$.

Master Theorem Direct Application

- For **$aT(n/b)+f(n)$** : compute **$n^{\log_b a}$** , compare with $f(n)$, pick case 1/2/3.

Counting Comparisons/Swaps

- Selection sort comparisons = **$n(n-1)/2$** , swaps = **$n-1$** .
- Bubble sort worst swaps = **$n(n-1)/2$** .

Stability/In-Place Property

- Stable: **Bubble, Insertion, Merge, Counting, Radix**; In-place $O(1)$: **Bubble, Selection, Insertion, Heap**.

Tracing Dijkstra/Bellman-Ford/Floyd-Warshall

- Practice small 4-6 vertex graphs; track distance and predecessor arrays per iteration.

MST Construction Order

- Kruskal adds edges in **weight order** skipping cycles; Prim grows from a vertex by cheapest crossing edge.

Greedy vs DP Applicability

- Greedy: **fractional knapsack, activity selection, Huffman, MST, Dijkstra**.
- DP: **0/1 knapsack, LCS, edit distance, matrix chain, coin change**.

Asymptotic Ordering

- **$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$** .

Hash Table Probe Sequence

- Linear: **$(h+i) \bmod m$** ; quadratic: **$(h+c_1 i+c_2 i^2) \bmod m$** ; double: **$(h_1+i h_2) \bmod m$** .

Final High-Yield Memory Hooks

- **Stable sorts mnemonic: "BIM-CR"** (Bubble, Insertion, Merge, Counting, Radix).
- **Greedy fails on 0/1 knapsack; use DP.**
- **Dijkstra fails on negative edges; use Bellman-Ford.**
- **Build-heap is $O(n)$; heapify-down is $O(\log n)$.**
- **Comparison sort lower bound = $\Omega(n \log n)$.**
- **Spanning tree edges = $V-1$; handshaking: sum of degrees = $2E$.**